Quantization as a Deployability Gate for Keyword Spotting on a Cortex-M4 MCU

Abstract

Deploying keyword spotting (KWS) on Cortex-M4 microcontrollers requires post-training quantization (PTQ). In practice, however, embedded engineers often discover toolchain incompatibilities and memory overruns only after attempting on-device deployment—long after model selection is complete. This gap matters directly for low-power, edge-intelligence system design. We present a controlled, physically measured comparison of four TensorFlow Lite PTQ variants: FP32, float16, dynamic-range INT8, and full-integer INT8. We apply them to a DS-CNN-S keyword-spotting model trained on Google Speech Commands v2, and deploy each variant to an STM32 Nucleo-F446RE (Cortex-M4, 180 MHz, 128 KiB SRAM) using STM32CubeIDE and X-CUBE-AI. Static analysis shows that FP32 (256.70 KiB RAM) and dynamic-range INT8 (194.46 KiB RAM) both exceed the board's SRAM budget. Float16 is unsupported by the toolchain for this target family. This leaves full-integer INT8 as the only deployable variant. It reaches 92.25% test accuracy, essentially matching the 92.18% FP32 reference (a difference of just 0.07 percentage points, well inside the sub-0.4-point spread across all four variants), with 62.26 KiB flash, 69.53 KiB RAM, 158.72 ms measured latency, and 4.39 ± 0.02 mJ measured energy per inference (INA238-based, averaged over three capture repetitions). Our central finding is that, on this board, PTQ scheme selection acts as a binary deployability gate rather than a continuous accuracy/efficiency trade-off. Three of the four standard TFLite PTQ options are inadmissible outright, and the sole survivor pays essentially no accuracy penalty for its footprint and energy savings. We argue that PTQ evaluations for MCU-class edge systems should report on-device feasibility alongside accuracy from the outset, not as an afterthought. We also discuss what this deployability-gate finding means for low-power system designers working under similarly tight SRAM budgets.

CCS Concepts

• **Hardware** → **Power estimation and optimization**; **Platform power issues**; • **Computing methodologies** → *Neural networks*; • **Computer systems organization** → *Embedded systems.*

Keywords

TinyML, keyword spotting, post-training quantization, edge AI, Cortex-M4, energy-efficient inference, low-power systems

1 Introduction

Always-on voice interfaces increasingly run their wake-word detection directly on ultra-low-power microcontrollers instead of sending raw audio to the cloud. This shift is driven by privacy, latency, and battery-life requirements. Keyword spotting (KWS) means recognizing a small vocabulary of spoken commands from a continuous audio stream. It is the canonical workload for this class of edge-intelligence deployment. Compact convolutional architectures such as the depthwise-separable DS-CNN family [21] have become the standard small-footprint baseline for it. Fitting such a model into the kilobytes-to-low-hundreds-of-kilobytes of RAM and flash on an ARM Cortex-M4-class device is difficult, and keeping per-inference latency and energy low enough for battery operation adds a second constraint. Post-training quantization (PTQ) is what makes this possible: it converts a trained floating-point model's weights, and optionally its activations, to lower-precision representations without retraining [8]. TensorFlow Lite offers several PTQ variants for this purpose: float16 weight quantization, dynamic-range INT8 quantization (INT8 weights with float activations computed at run time), and full-integer INT8 quantization (INT8 weights and activations, calibrated against a representative dataset). Each variant sits at a different point on the accuracy/footprint/speed trade-off, alongside the unquantized FP32 model, which serves as an accuracy ceiling.

Quantization for embedded neural networks has been studied extensively on Cortex-M-class cores [7, 12], and more broadly on GPUs, FPGAs, and ASIC accelerators [1, 4, 5, 9, 13]. Within this body of work, however, one specific comparison is missing: a controlled, apples-to-apples evaluation of FP32, float16, dynamic-range INT8, and full-integer INT8 on the same trained model, the same dataset, and the same physical Cortex-M4 board, reporting accuracy, latency, RAM, flash, and energy together. This is exactly the comparison a practitioner needs when choosing among TFLite's PTQ options for a KWS deployment. As Section 2 details, existing studies fall into two groups. One group measures real hardware behavior, but for a different application, a different compression technique, or only a subset of these five metrics [2, 3, 6, 7, 15, 19]. The other group addresses KWS with exactly this PTQ vocabulary, but relies on desktop simulation, cost models, or figures drawn from other papers rather than physical measurement [11, 16, 17].

This paper closes that gap with a single, controlled study framed around the low-power, resource-constrained embedded-systems concerns central to this venue's Low Power Systems and Edge Intelligence tracks. We train a DS-CNN-S keyword-spotting model on the 12-class Google Speech Commands v2 [18] benchmark, export it as four variants (FP32, PTQ float16, PTQ dynamic-range INT8, and PTQ full-integer INT8), and deploy each one on an STM32 Nucleo-F446RE (Cortex-M4, 180 MHz) using STM32CubeIDE and X-CUBE-AI's Analyze and Validate-on-target workflow.

The contributions of this paper are:

- We present a controlled, single-pipeline, four-way PTQ comparison for keyword spotting on a real Cortex-M4 target
- We provide a jointly reported accuracy–latency–memory–energy trade-off table, entirely measured rather than estimated, for each scheme
- We show that the accuracy/footprint Pareto front among these variants is dominated by the deployability gate itself: the scheme that is Pareto-optimal on accuracy and footprint alone is eliminated before that trade-off ever becomes actionable for resource-constrained KWS deployment

- We evaluate the practical implications of a hard deployability gate for engineers designing low-power embedded and edge-intelligence systems more broadly, beyond keyword spotting specifically.

2 Related Work

Quantization and compression on microcontroller-class cores: Grimaldi et al. [7] jointly apply structured filter pruning and post-training quantization to compact CNNs, including a KWS model on the same Speech Commands benchmark we use, and physically measure RAM footprint and inference latency on Cortex-M4/M7 boards. Their quantization scheme, however, is a generic n-bit scheme, not the FP16, dynamic-range, or full-integer INT8 variants exposed by standard TFLite tooling, and they do not report flash footprint or energy. Peluso et al. [12] similarly evaluate quantization on Cortex-M devices across three tasks including KWS, but accuracy is obtained via simulation rather than on-device inference. Cerutti et al. [3] target the same sub-milliwatt KWS problem directly, combining an analog binary front-end with a binary neural network on a Cortex-M4 and a GAP8 RISC-V core, and report measured energy per inference, including a Hello Edge DS-CNN baseline figure of 10.52 mJ that we use as an external reference point. Their extreme 1-bit quantization, however, is not among the standard PTQ formats we compare, and they do not report latency, RAM, or flash.

Physically measured energy on STM32 with the same toolchain: Several recent studies use STM32CubeIDE and X-CUBE-AI with external current-sense measurement, the methodology we adopt, for other applications and compression techniques. Tao et al. [15] measure accuracy, latency, a combined weight-and-activation model size, and energy for QAT and unstructured pruning of a ResNet-18 bird-call classifier; Ben Khalifa and Martel [2] measure latency and energy for a custom SMT-derived fixed-point quantizer on small classifiers; Xu et al. [19] measure power and energy for QAT and pruning across several architectures, including DS-CNN, for hand-gesture recognition; and Govarthan et al. [6] measure latency and energy for 8-bit-quantized incremental-learning models for activity and environmental sensing. None applies this measurement rigor to a multi-scheme PTQ sweep for keyword spotting.

Post-training quantization schemes for keyword spotting. Triesh and Ahmed [16] name the exact dynamic-range and full-integer quantization schemes we compare, and report accuracy, latency, RAM, flash, and energy for KWS across TFLite Micro, Edge Impulse, and uTensor, but by their own account these figures come from reference literature rather than independent measurement. Vummaneni et al. [17] model INT8 quantization, pruning, and knowledge distillation for a DS-CNN KWS model across a range of Cortex-M cores using a calibrated cost model, explicitly cautioning that "results are simulation-driven . . . treat absolute numbers as illustrative." Lovison Franco et al. [11] compare FP32, 8-bit QAT, and binary networks for KWS on Speech Commands v2 entirely in a desktop TensorFlow/Larq simulation, reporting only model size.

This picture is reinforced by broader, non-MCU quantization studies on larger edge accelerators [1, 4, 5, 9, 13], a Cortex-M4 study reporting accuracy and flash alone [20], a simulated multi-objective compression pipeline [14], and a general TinyML compression survey [10]. Across this literature, a clear methodological split persists: studies either rely on desktop simulation rather than physical MCU execution, or measure isolated metrics while omitting critical dimensions such as on-device RAM, flash, or energy. No existing work provides a controlled, physically measured, four-way PTQ comparison across accuracy, latency, RAM, flash, and energy on a single Cortex-M4 target. Table 1 summarizes how these disparate efforts compare across the five metrics we report together, highlighting the gap this paper closes.

3 Methodology

Fig. 1 summarizes the complete experimental pipeline described in this section, from raw audio to the final measured energy figure. (1) Utterances from Google Speech Commands v2 are collected under the 12-class protocol. (2) Each utterance is converted to a $49 \times 40 \times 1$ MFCC feature map. (3) These features train a single DS-CNN-S model to convergence, producing one FP32 checkpoint. (4) From this checkpoint, TFLiteConverter exports four PTQ variants: FP32, float16, dynamic-range INT8, and full-integer INT8. (5) Each variant is passed through X-CUBE-AI's static *Analyze* stage, which checks RAM, flash, and tensor-type support against the Nucleo-F446RE target: FP32 and dynamic-range INT8 are eliminated for exceeding the 128 KiB SRAM budget, and float16 is eliminated because the toolchain does not support its tensor type, leaving full-integer INT8 as the only variant that clears this static gate. (6) The surviving variant is flashed to the board for *Validate-on-target* correctness and latency checking, then subjected to INA238-based energy measurement. The remainder of this section details each of these six steps.

Dataset and features: We use the Google Speech Commands v2 [18] corpus under the standard 12-class protocol: the ten target keywords (*yes, no, up, down, left, right, on, off, stop, go*), a pooled *unknown* class covering the corpus's other spoken words, and a *silence* class. Utterances are assigned to train/validation/test splits using the corpus's official file lists, so no speaker's recordings cross a split boundary. The *unknown* class is downsampled per split to roughly the size of one keyword class, so it does not dominate training. *Silence* clips are synthesized by cropping one-second, 16 kHz windows at random offsets from the corpus's background-noise recordings. Each one-second clip is converted to a 40-dimensional MFCC feature map. A short-time Fourier transform (40 ms frame length, 20 ms hop, 1024-point FFT) is projected onto 40 mel bins over 20 Hz–4 kHz, log-compressed, and converted to 40 MFCC coefficients per frame. This yields a $49 \times 40 \times 1$ feature map per utterance.

Fig.2 illustrates this front-end transformation across three representative acoustic inputs: a target keyword (*yes*), an out-of-vocabulary word (*bed*), and synthesized ambient silence.

Model: We use DS-CNN-S, the smallest depthwise-separable convolutional architecture from the Hello Edge keyword-spotting family [21]. The network starts with a strided standard convolution (64 filters, 10×4 kernel, stride 2×2), followed by four depthwise-separable blocks (a 3×3 depthwise convolution and a 1×1 pointwise convolution to 64 channels, each followed by batch normalization and ReLU). It ends with global average pooling and a 12-way softmax classifier. Every convolution is bias-free; batch normalization absorbs the bias term instead. This keeps the model small and makes

Table 1: How prior work compares along the five metrics this paper reports together (accuracy, latency, RAM, flash, energy), physically measured, for keyword spotting on a single Cortex-M4 board. ✓ = reported; – = not reported; † = simulated or borrowed rather than independently measured on that hardware; ‡ = a combined model-size figure rather than an on-device flash/RAM split from a static analysis pass. RAM and Flash are shown as separate columns because several studies report only one of the two.

Study	Hardware	Task	Acc.	Latency	RAM	Flash	Energy	Phys. meas.
Grimaldi et al. [7]	Cortex-M4 / M7	KWS (+ other)	✓	✓	✓	–	–	✓
Peluso et al. [12]	Cortex-M (sim.)	KWS (+ other)	✓†	–	–	–	–	–†
Cerutti et al. [3]	Cortex-M4 + GAP8	KWS	✓	–	–	–	✓	✓
Tao et al. [15]	STM32H7A3	Bird-call (QAT+prune)	✓	✓	✓‡	✓‡	✓	✓
Ben Khalifa & Martel [2]	Cortex-M3	Small classifiers	–	✓	–	–	✓	✓
Xu et al. [19]	ESP32-S3 / Cortex-M4F	Hand gesture (QAT+prune)	✓	–	–	–	✓	✓
Govarthan et al. [6]	Cortex-M7 / M4	Activity / env. sensing	✓	✓	–	–	✓	✓
Triesh & Ahmed [16]	TFLite Micro / EI / uTensor	KWS	✓†	✓†	✓†	✓†	✓†	–†
Vummaneni et al. [17]	Cortex-M (cost model)	KWS	✓†	✓†	✓†	✓†	✓†	–†
Lovison Franco et al. [11]	Desktop (sim.)	KWS	✓	–	–‡	–‡	–	–†
This work	**Cortex-M4 (Nucleo-F446RE)**	**KWS**	✓	✓	✓	✓	✓	✓

‡Reports a combined weight-and-activation model-size figure only, not an on-device RAM/flash split from a static analysis pass.

it directly amenable to full-integer quantization. The model has approximately 24K trainable parameters, consistent with the "small" DS-CNN configuration reported in the original Hello Edge study.

Post-training quantization variants: From a single trained FP32 checkpoint we export four `.tflite` variants with `TFLiteConverter` : (1) the unquantized **FP32** model, used as an accuracy reference and not intended for deployment; (2) **PTQ float16**, which stores weights in IEEE half precision and dequantizes them to float32 at inference time; (3) **PTQ dynamic-range INT8**, which quantizes weights to INT8 ahead of time while computing activations in float32 at run time, requiring no calibration data; and (4) **PTQ full-integer INT8**, which quantizes both weights and activations to INT8 using scale/zero-point parameters calibrated against a representative dataset [8], with INT8 tensors set as the model's native input/output type, so no on-device float-to-int conversion is needed. The representative dataset for calibration is drawn exclusively from the training split (300 single-sample draws). This keeps the held-out test and validation sets uncontaminated by the quantization procedure.

Target platform and toolchain: All four variants are imported into STM32CubeIDE with the X-CUBE-AI expansion pack (ST Edge AI Core v2.2.0, X-CUBE-AI 10.2.1) targeting an STM32 Nucleo-F446RE board (Arm Cortex-M4 core at 180 MHz, 512 KiB flash, 128 KiB SRAM). X-CUBE-AI's *Analyze* stage performs a static analysis of each imported model. It reports the flash footprint of the generated network code and weights, and the RAM required for activation buffers and the runtime's internal state, all without requiring board access. This static pass is what first exposes the RAM-budget and tensor-type-support constraints discussed in Section 5. Variants that pass Analyze are then flashed to the board and exercised with X-CUBE-AI's *Validate-on-target* stage. This stage feeds a batch of test vectors through the on-device network, cross-checks the outputs numerically against a host-computed reference to confirm correct deployment, and reports profiled per-inference latency.

Energy measurement protocol: For the variant(s) that deploy successfully, we measure energy per inference with an INA238 digital power monitor (15 mΩ shunt) placed in series with the Nucleo board's target-MCU supply rail. This is the dedicated on-board current-measurement jumper that isolates the STM32's own draw from the ST-LINK debug circuitry. Fig. 4 shows the full measurement chain, from the target supply rail to the host PC's CSV log. We sample bus voltage and current over I^2C at approximately 45 Hz from an ESP32-S3 logging host, which runs independently of the STM32 itself. On each board reset, firmware on the Nucleo holds an *idle* state for 15 s with no inference activity, then runs 200 back-to-back inferences as an *active* burst. Running many inferences back-to-back, rather than one, averages out per-sample timing jitter in the power trace. The board's on-board LED flashes once at the idle-to-active transition and twice at burst completion, so the boundary is visible on an oscilloscope or by eye, without adding an extra measurement wire. We compute average power separately over the idle window, P_{idle}, and the active window, P_{active}. This is bus voltage times current, computed in software rather than taken from the sensor's internal power register, with a half-second margin trimmed from each window edge to exclude transition effects. Energy per inference is then:

$$E_{\text{inf}} = (P_{\text{active}} - P_{\text{idle}}) \times t_{\text{lat}}, \tag{1}$$

Where, t_{lat} is the per-inference latency obtained from Validate-on-target. Fixed idle overhead, such as clock generation and peripheral leakage, is excluded this way, so that only the marginal cost of running inference is attributed to the model.

4 Experimental Setup

We train the FP32 baseline with the Adam optimizer (initial learning rate 10^{-3}), sparse categorical cross-entropy loss, and a batch size of 64, for up to 40 epochs. We use early stopping with a patience of 8 epochs on validation accuracy, restoring the best-observed weights, and reduce the learning rate by 0.5× after 4 stagnant epochs. Table 2 lists these training hyperparameters in full, so the training run can be reproduced exactly. All four PTQ variants are exported from this single converged checkpoint, so any accuracy difference between variants reflects the quantization scheme alone rather than training variance. We evaluate desktop accuracy for every variant with the TFLite interpreter on the full, held-out test split

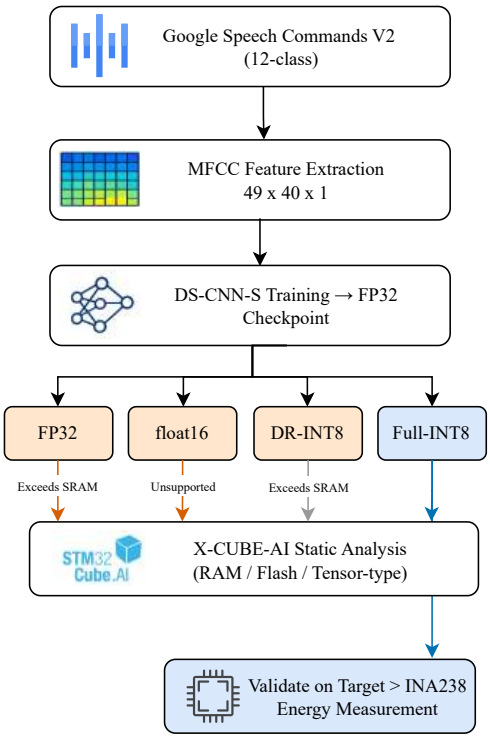


Figure 1: High-level overview of the experimental pipeline: dataset and feature extraction, model training, four-way PTQ export, X-CUBE-AI static feasibility analysis (the deployability gate), and on-device validation with energy measurement.

(4,888 utterances), handling both floating-point and INT8-quantized input/output tensors through the same evaluation routine.

Table 2: Training hyperparameters for the FP32 checkpoint shared by all four PTQ exports

Hyperparameter	Value
Optimizer	Adam
Initial learning rate	10^{-3}
LR reduction	0.5× after 4 stagnant epochs
Loss	Sparse categorical cross-entropy
Batch size	64
Max. epochs	40
Early-stopping patience	8 epochs (on val. accuracy)
Weight restore	Best validation checkpoint

We clock the Nucleo-F446RE at its maximum rated 180 MHz (HSI oscillator, PLL, Over-Drive mode, voltage scaling range 1) for both the Analyze/Validate and energy measurements. All reported figures therefore reflect the board's maximum-performance operating point, not a reduced-clock, lower-power configuration. The full-integer INT8 variant's activation (tensor-arena) buffer is statically sized at 70 KiB, matching the 69.53 KiB Analyze reported as required with a small safety margin. We measure energy per inference over three independent capture repetitions, each a full board reset, 15 s idle window, and 200-inference active burst, to check run-to-run repeatability. We report the mean and sample standard deviation across these three repetitions, rather than a single measurement.

5 Results and Discussion

Table 3 reports accuracy, flash, RAM, latency, and energy for all four variants. Two findings emerge before latency or energy enters the picture at all. First, X-CUBE-AI's Analyze stage shows that FP32 (256.70 KiB) and dynamic-range INT8 (194.46 KiB) both exceed the Nucleo-F446RE's 128 KiB SRAM budget. Neither can be flashed to this board, regardless of how accurate or fast it might be. Second, PTQ float16 fails at the Analyze stage itself: ST Edge AI Core v2.2.0 (X-CUBE-AI 10.2.1) does not support the FLOAT16 tensor type for the stm32f4 target family and raises an internal error before any footprint figure can be produced. Only the fourth variant, full-integer INT8, satisfies both constraints and is the sole variant carried through to Validate-on-target and energy measurement. Fig. 5 makes this gate visual: two of the four bars clear the board's SRAM ceiling outright, and one variant has no bar at all because the toolchain cannot even analyze it.

This is the paper's central finding. For this model and this 128 KiB-RAM Cortex-M4 target, *the choice of PTQ scheme is not a tunable efficiency knob that trades a little accuracy for a little footprint. It is a binary deployability gate.* Three of the four standard TensorFlow Lite PTQ options considered here are inadmissible on this hardware/toolchain combination, independent of their accuracy. A practitioner who evaluates PTQ schemes only on desktop accuracy and model-file size, without an on-device static analysis pass, would not discover this until failing to flash the board.

Accuracy itself is essentially unaffected by quantization aggressiveness: 92.18% (FP32) vs. 92.23% (float16) vs. 92.57% (dynamic-range INT8) vs. 92.25% (full-integer INT8) on the 4,888-utterance test set. This is a spread of under 0.4 percentage points. The most aggressively quantized deployable variant, full-integer INT8, loses nothing relative to the FP32 ceiling. For a small-vocabulary, low-capacity model like DS-CNN-S, the 16× reduction in numeric precision from FP32 to INT8 evidently costs far less accuracy than it saves in footprint. Relative to the FP32 reference, full-integer INT8 cuts flash from 110.45 KiB to 62.26 KiB (44% smaller) and RAM from 256.70 KiB, which is infeasible on this board, to 69.53 KiB.

Setting deployability aside, Table 3 also lets us ask which variant is Pareto-optimal on accuracy and footprint alone. Dynamic-range INT8 has the highest accuracy of any variant (92.57%), while full-integer INT8 has the smallest flash and RAM footprint; neither dominates the other, so both are Pareto-optimal, and FP32 is dominated outright since full-integer INT8 matches or beats it on every axis. This is exactly the trade-off a purely accuracy/efficiency-driven search would explore. The static feasibility gate makes that exploration moot here: dynamic-range INT8 is eliminated for exceeding

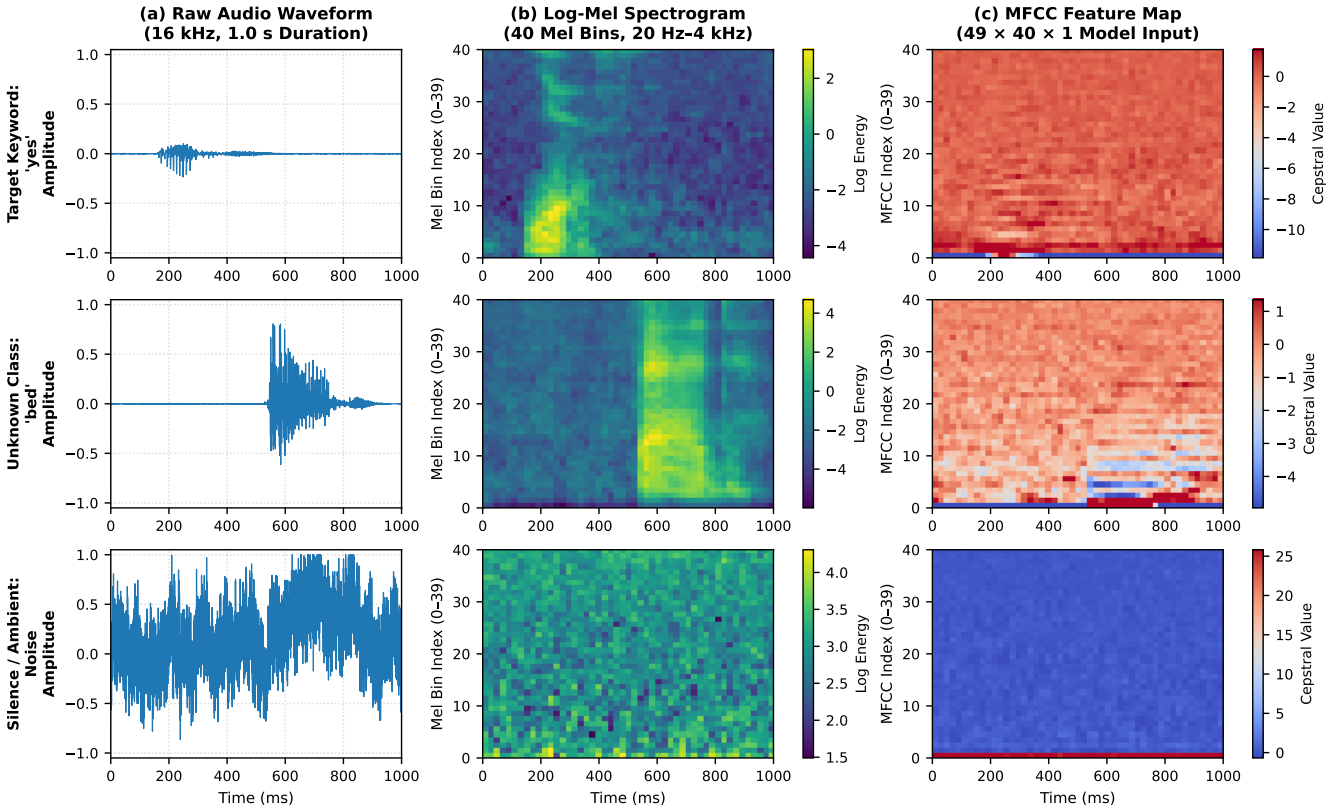


Figure 2: Three-stage acoustic feature extraction pipeline illustrated across three representative classes (Target Keyword *yes*, Unknown class *bed*, and synthesized ambient silence): (a) raw 16 kHz time-domain waveform (1.0 s duration), (b) 40-band log-mel filterbank spectrogram (20 Hz–4 kHz), and (c) the resulting $49 \times 40 \times 1$ MFCC input tensor ingested by the DS-CNN-S model on the Cortex-M4 MCU.

the 128 KiB SRAM budget before its accuracy advantage can ever be exercised on this board, leaving full-integer INT8 as the sole realizable point on what would otherwise be a two-point Pareto front. This is the concrete sense in which the deployability gate dominates the accuracy/footprint trade-off, rather than merely coexisting with it.

On the deployed full-integer INT8 variant, Validate-on-target measures a mean per-inference latency of 158.72 ms at 180 MHz. The INA238-based protocol of Section 3 measures $P_{\text{idle}} = 121.69 \pm 0.15$ mW and $P_{\text{active}} = 149.34 \pm 0.08$ mW, mean ± sample standard deviation across three independent capture repetitions. Because P_{idle} and P_{active} are measured within the same three repetitions rather than independently, we compute the marginal inference power directly from the three paired per-repetition differences rather than by propagating the marginal standard deviations; this gives 27.65 ± 0.12 mW and, via Equation (1), an energy cost of 4.39 ± 0.02 mJ per inference. The spread across the three repetitions is tight, under 0.5% of the mean for both power terms. This indicates that the board's power draw during this workload is highly repeatable, rather than dominated by measurement noise. Fig. 6 shows one representative capture cycle, with the idle and active averaging windows shaded and P_{idle}, P_{active}, and their difference called out directly on the trace. For context, Cerutti et al. report 10.52 mJ per inference for a Hello Edge DS-CNN baseline on comparable Cortex-M4-class hardware [3]. Our full-integer INT8 figure is lower. This is consistent with INT8 arithmetic drawing less switching current per multiply-accumulate than the floating-point or binary-network execution paths used in that comparison. The two measurements are not on identical model architectures or boards, though, so this should be read as a rough sanity check rather than a controlled comparison.

Taken together, these results show that for MCU-class keyword spotting under a tight SRAM budget, full-integer INT8 is not merely a reasonable choice among several—on this board it is the *only* choice that runs at all, and it does so at essentially no accuracy cost relative to alternatives that cannot be deployed at all.

To verify that 8-bit integer quantization does not introduce class-specific degradation or bias, Fig. 7 plots the normalized confusion matrix for the deployed full-integer INT8 model evaluated on the 4,888 test utterances. The model demonstrates robust sensitivity across all 10 target keywords, with individual recalls ranging from 88.1% (*on*) to 96.7% (*yes*). Ambient silence achieves 100.0% specificity (407/407), guaranteeing zero false triggers during quiescent background periods. Minor confusions between acoustically similar

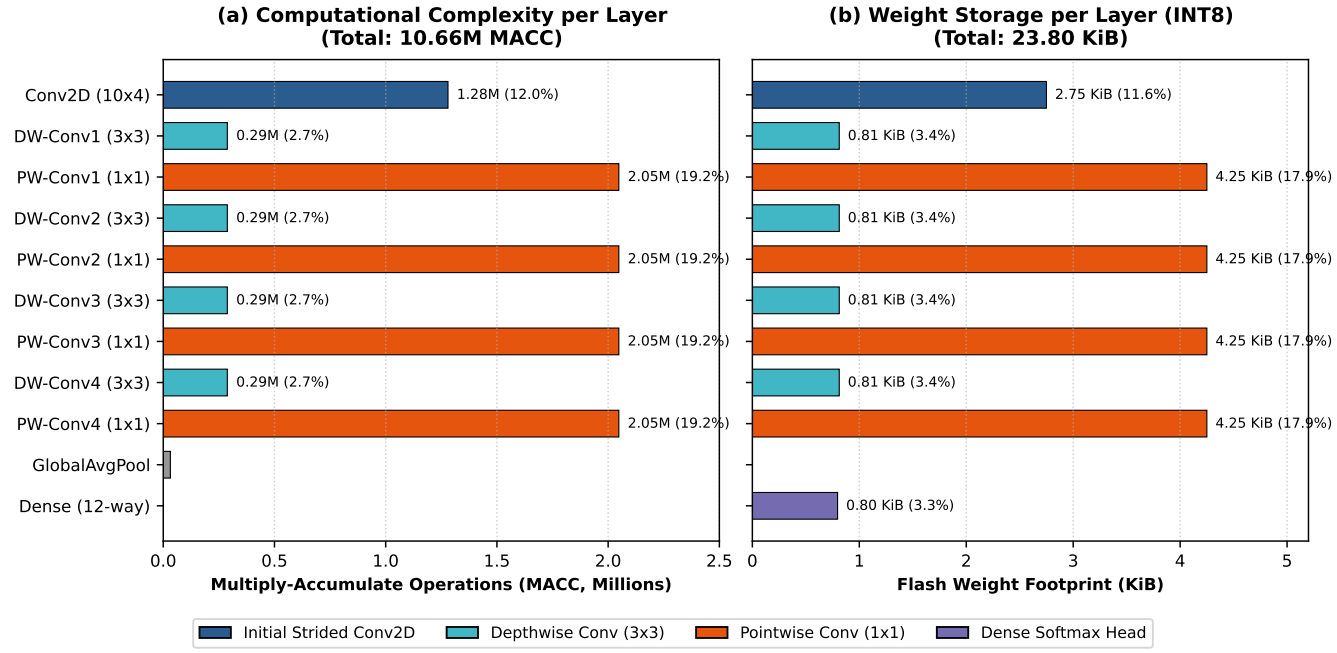


Figure 3: Layer-wise architectural profiling of DS-CNN-S from ST Edge AI Core (X-CUBE-AI): (a) arithmetic complexity in multiply-accumulate operations (totaling 10.66 M MACC), and (b) on-device flash weight footprint in INT8 precision (totaling 23.80 KiB). Pointwise 1×1 projections account for 76.9% of arithmetic operations and 71.4% of weight storage, while 3×3 depthwise convolutions incur minimal compute overhead (2.7% MACC each).

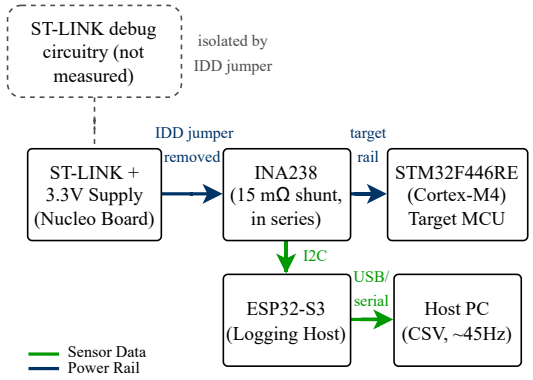


Figure 4: Energy measurement setup. The Nucleo's current-measurement jumper is replaced with an INA238 (15 mΩ shunt) in series with the STM32 target MCU's own supply rail, isolating it from the ST-LINK debug circuitry. An ESP32-S3 samples the INA238 over I^2C and logs to a host PC as CSV.

pairs (such as *no* vs. *go* at 4.5%, and *up* vs. *off* at 3.2%) misclassifications into the open *unknown* category remain below 3% for all target keywords.

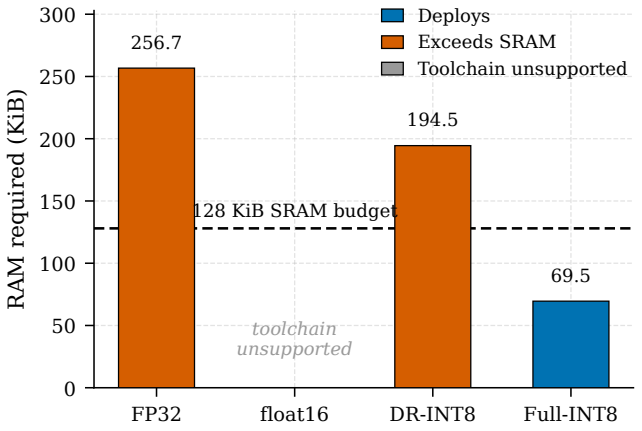


Figure 5: RAM required by each PTQ variant (X-CUBE-AI Analyze) against the Nucleo-F446RE's 128 KiB SRAM budget. Only full-integer INT8 fits.

5.1 Implications for Low-Power and Edge-Intelligence System Design

Although this study is framed around one model and one board, the underlying pattern generalizes to a concern central to low-power embedded system design more broadly. As SRAM budgets shrink toward the kilobytes-to-low-hundreds-of-kilobytes range typical

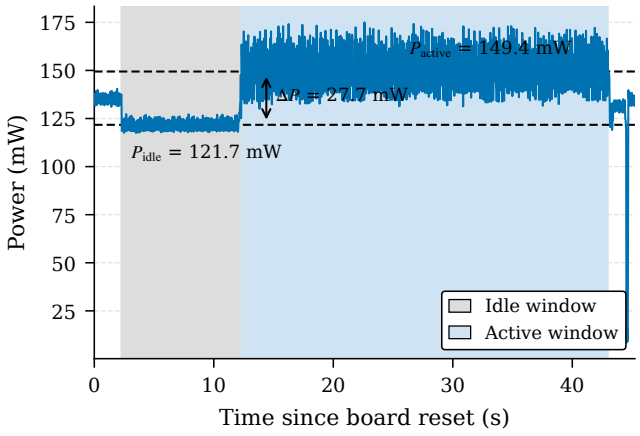


Figure 6: One representative idle-to-active measurement cycle for the full-integer INT8 variant (INA238, ∼45 Hz). Shaded regions mark the averaging windows used in Eq. (1); edges are trimmed by 0.5 s before averaging to exclude transition effects.

Table 3: Accuracy, footprint, latency, and energy across PTQ variants on the STM32 Nucleo-F446RE (128 KiB SRAM, 512 KiB flash).

Variant	Acc. (%)	Flash (KiB)	RAM (KiB)	Lat. (ms)	Energy (mJ)
FP32	92.18	110.45	256.70†	N/A	N/A
float16	92.23	N/A‡	N/A‡	N/A	N/A
DR-INT8	92.57	107.21	194.46†	N/A	N/A
Full-INT8	92.25	62.26	69.53	158.72	4.39 ± 0.02

†**Infeasible:** Exceeds target's 128 KiB SRAM limit (static report); cannot be flashed to board.
‡**Unsupported:** ST Edge AI Core v2.2.0 lacks `FLOAT16` tensor support for `stm32f4`; static analysis fails. Accuracy measured via desktop TFLite interpreter.

of Cortex-M4-class edge nodes, a growing share of a workload's design space is not merely *suboptimal* but *infeasible*, and standard tooling does not always surface that distinction early. In this study, three of four industry-standard PTQ options failed a static analysis pass before any accuracy or latency question was even relevant. For system designers building energy-aware or power-constrained edge-intelligence pipelines, this suggests a concrete practice: run a static feasibility check, meaning flash and RAM footprint against the target's budget and toolchain tensor-type support, as the first filtering step in a compression-scheme search, before spending board time or energy-measurement effort on any candidate. Treating on-device feasibility as a first-class, binary constraint, rather than folding it into a continuous accuracy/efficiency score, turns such a search into a two-stage process: (1) eliminate infeasible schemes via static analysis, then (2) optimize among the survivors on accuracy, latency, and energy. For our model and board, that survivor set had exactly one member; the two-stage framing itself is the reusable takeaway for practitioners working with different models, boards, or compression techniques within the same resource envelope.

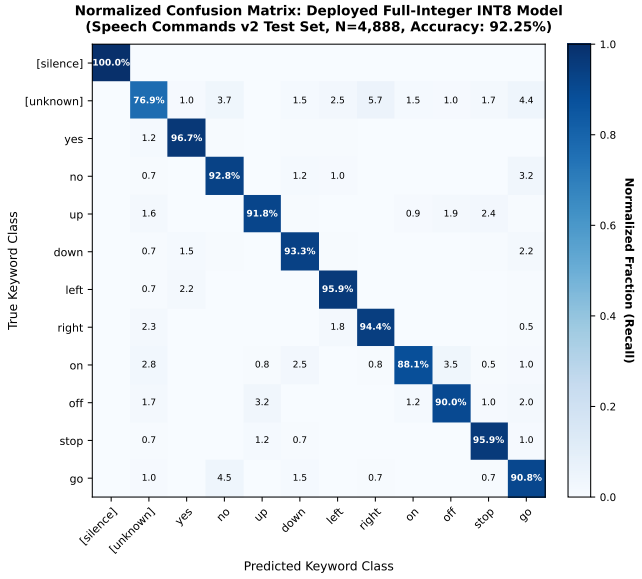


Figure 7: Normalized confusion matrix of the full-integer INT8 DS-CNN-S model deployed on the target MCU, evaluated on the official Speech Commands v2 test split (N = 4,888, overall accuracy 92.25%). Diagonal entries denote per-class sensitivity (recall); ambient silence is recognized with 100.0% specificity, ensuring zero spurious wake-ups, while per-keyword recalls remain tightly clustered between 88.1% and 96.7%.

This framing extends beyond keyword spotting to the broader class of battery-powered, memory-constrained systems this venue's Low Power Systems and Edge Intelligence tracks target: a wearable health monitor classifying accelerometer data, a solar-powered environmental sensor running an anomaly detector, or an industrial vibration sensor doing on-device fault classification all share the same constraint—a fixed, small SRAM and flash budget that a compression scheme either fits inside or does not. In each setting, the same static feasibility gate applies before accuracy or energy enters the discussion, and the same two-stage search (eliminate the infeasible schemes, then optimize among survivors) carries over, even though the model, sensor, and task differ from ours. We see this as the more general contribution of this study: not a single number for one board, but a reusable way to think about compression search under a hard memory ceiling.

5.2 Scope Boundaries and Generalizable Lessons

Our findings operate within specific scope boundaries. First, they reflect a single model architecture (DS-CNN-S) and a specific target board (STM32 Nucleo-F446RE): the exact RAM thresholds where FP32 and dynamic-range INT8 become infeasible, and the point where float16 support fails, are governed by this board's 128 KiB SRAM budget and this toolchain version (ST Edge AI Core v2.2.0 / X-CUBE-AI 10.2.1). These thresholds are properties of this board and toolchain version, not universal constants.

Second, our energy measurement isolates the STM32 target MCU's own supply rail via the board's current-measurement jumper, excluding the ST-LINK debug circuitry and peripheral sensor front-ends, so the reported figures capture the model-inference component of the system budget rather than a total system-level draw. Third, our protocol shows high repeatability across three repetitions on a single physical unit but does not quantify board-to-board manufacturing variation. Finally, using a single trained FP32 checkpoint as the common ancestor for all four PTQ variants isolates the quantization effect cleanly but reflects one training run rather than a distribution across seeds; given the sub-0.4-point accuracy spread observed, this is unlikely to change our qualitative conclusions, though it leaves training-seed robustness under tight memory caps for future work.

6 Conclusion

We presented a controlled, physically measured comparison of four post-training quantization schemes (FP32, float16, dynamic-range INT8, and full-integer INT8) for a DS-CNN-S keyword-spotting model deployed to an STM32 Nucleo-F446RE. We reported accuracy, flash, RAM, latency, and INA238-measured energy per inference together on a single Cortex-M4 target. Our central finding is that PTQ scheme choice functions here as a binary deployability gate, not a smooth accuracy/efficiency dial. Two variants exceed the board's 128 KiB SRAM budget, one is unsupported by the deployment toolchain for this target family, and only full-integer INT8 both fits and runs, at 92.25% accuracy, 158.72 ms latency, and 4.39 ± 0.02 mJ per inference, with negligible accuracy cost relative to the FP32 ceiling. We recommend that PTQ evaluations for microcontroller-class targets report on-device feasibility alongside accuracy from the outset, rather than assuming every TensorFlow Lite PTQ option is a viable candidate. We also suggest that low-power system designers treat static feasibility analysis as a first-stage filter, ahead of any accuracy- or energy-driven compression search (Section 5.1). Future work includes three directions. One is extending this comparison with quantization-aware training at sub-8-bit and mixed-precision widths, to test whether shrinking the representation further, which standard 8-bit PTQ cannot do, can pull additional variants under the SRAM ceiling while preserving accuracy. Another is repeating the study on a board with a larger SRAM budget and a newer toolchain release, to test whether the specific gates observed here shift accordingly. A third is cross-validating latency and energy on a second architecture, such as an ESP32-S3 target.

Ethics and Privacy Statement

This study trains and evaluates on Google Speech Commands v2 [18], a public, de-identified benchmark of short spoken-word recordings released for research use; we collect no new human-subject data and record no identifying information about any speaker. All hardware measurements are taken on our own boards with no third-party systems or human participants involved, and we are not aware of any dual-use risk beyond those already inherent to keyword-spotting technology.

References

[1] Oumayma Bel Haj Salah, Seifeddine Messaoud, Mohamed Ali Hajjaji, Mohamed Atri, and Noureddine Liouane. 2025. Efficient Post-Training Quantization for FPGA-Based Neural Networks. In *Proc. Int. Conf. Control, Automation and Diagnosis (ICCAD)*.

[2] Dorra Ben Khalifa and Matthieu Martel. 2024. Rigorous Floating-Point to Fixed-Point Quantization of Deep Neural Networks on STM32 Micro-controllers. In *Proc. 10th Int. Conf. Control, Decision and Information Technologies (CoDIT)*.

[3] Gianmarco Cerutti, Lukas Cavigelli, Renzo Andri, Michele Magno, Elisabetta Farella, and Luca Benini. 2022. Sub-mW Keyword Spotting on an MCU: Analog Binary Feature Extraction and Binary Neural Networks. *IEEE Trans. Circuits Syst. I, Reg. Papers* (2022).

[4] Geunjae Choi, Kamin Lee, and Nojun Kwak. 2025. WAPS-Quant: Low-Bit Post-Training Quantization Using Weight-Activation Product Scaling. *IEEE Access* (2025).

[5] Satish Kumar Dubey and Naeem Ahmad. 2025. A Multi Stage Framework for Efficient Deep Neural Network Compression on Edge Devices. In *Proc. 13th Int. Conf. Intelligent Systems and Embedded Design (ISED)*.

[6] V. Govarthan, M. Thangamani, R. Aarthi, S. Satheesh, M. Moorthy, and Kavitha V. Kakade. 2025. TinyML on Microcontrollers: Enabling Energy-Efficient, Real-Time, Privacy-Preserving Incremental Learning for Embedded Systems. In *Proc. Int. Conf. Sustainable Communication Networks and Application (ICSCN)*.

[7] Matteo Grimaldi, Valentino Peluso, and Andrea Calimera. 2019. Optimality Assessment of Memory-Bounded ConvNets Deployed on Resource-Constrained RISC Cores. *IEEE Access* (2019).

[8] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*.

[9] Yumi Kim, Kyungho Kim, Dongyeob Shin, Sung-Joon Jang, and Sang-Seol Lee. 2025. Fully Integer Post-Training Quantization in LiDAR-Based 3D Object Detection for Efficient Hardware Acceleration. In *Proc. 25th Int. Conf. Control, Automation and Systems (ICCAS)*.

[10] Ismail Lamaakal, Chaymae Yahyati, Ibrahim Ouahbi, Khalid El Makkaoui, and Yassine Maleh. 2025. A Survey of Model Compression Techniques for TinyML Applications. In *Proc. Int. Conf. Circuit, Systems and Communication (ICCSC)*.

[11] Bruno Lovison Franco, Jonathan Miquel, Aymen Romdhane, Guillaume Prenat, Lorena Anghel, David Novo, and Pascal Benoit. 2024. Trade-Offs in Neural Network Compression: Quantized and Binary Models for Keyword Spotting. In *Proc. 31st IEEE Int. Conf. Electronics, Circuits and Systems (ICECS)*.

[12] Valentino Peluso, Matteo Grimaldi, and Andrea Calimera. 2019. Arbitrary-Precision Convolutional Neural Networks on Low-Power IoT Processors. In *Proc. IFIP/IEEE 27th Int. Conf. Very Large Scale Integration (VLSI-SoC)*.

[13] Rakandhiya D. Rachmanto, Zaki Sukma, Ahmad N. L. Nabhaan, Arief Setyanto, Ting Jiang, and In Kee Kim. 2024. Characterizing Deep Learning Model Compression with Post-Training Quantization on Accelerated Edge Devices. In *Proc. IEEE Int. Conf. Edge Computing and Communications (EDGE)*.

[14] Bidyut Saha, Riya Samanta, Soumya K. Ghosh, and Ram Babu Roy. 2025. GenCPruneX: Adaptive Channel-wise Pruning for Efficient TinyML Deployment with Genetic Multi-Objective Optimization. In *Proc. COMSNETS Poster Track*.

[15] Yonghui Tao, Yusuke Hioka, and Yuqian Lu. 2022. Experimental Energy Consumption Analysis of Neural Network Model Compression Methods on Microcontrollers with Applications in Bird Call Classification. In *Proc. IEEE Asia-Pacific Conf. Computer Science and Data Engineering (CSDE)*.

[16] Amel A. Triesh and Nuredin Ali Salem Ahmed. 2025. Embedded AI: Integrating TinyML in Resource-Constrained IoT Devices. In *Proc. IEEE Int. Multi-Conf. Smart Systems & Green Process (IMC-SSGP)*.

[17] Naga Sujitha Vummaneni, Adarsh Mittal, Ishan Kumar, Srilakshmi Bharadwaj, and Himani Varshney. 2026. Trend-Aware TinyML Co-Design on Microcontrollers: Quantization, Structured Sparsity, and On-Device Adaptation Under Tight Memory–Energy Budgets. In *Proc. IEEE SoutheastCon*.

[18] P. Warden. 2018. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. *ArXiv e-prints* (April 2018). arXiv:1804.03209 [cs.CL] https://arxiv.org/abs/1804.03209

[19] Yang Xu, Chaoping Zhu, and Yingqiang Wang. 2025. A Generalized TinyML Workflow for Energy-Efficient Hand Gesture Recognition on ESP32S3. In *Proc. 6th Int. Conf. Artificial Intelligence and Computer Engineering (ICAICE)*.

[20] Chaymae Yahyati, Ismail Lamaakal, Khalid El Makkaoui, Ibrahim Ouahbi, and Yassine Maleh. 2025. TinyML-Based Facial Recognition for Embedded Systems. In *Proc. Int. Conf. Circuit, Systems and Communication (ICCSC)*.

[21] Yundong Zhang, Naveen Suda, Liangzhen Lai, and Vikas Chandra. 2018. Hello Edge: Keyword Spotting on Microcontrollers. arXiv:1711.07128.